Running Persistent AI Worlds

Multi-Agent Simulations With Progressive Learning and Emergent Behavior

Compiled February 2026 · Deep Research Report

Covering: Open-Ended Learning · Multi-Agent RL · Artificial Life
Evolutionary Strategies · LLM Agent Simulations · Practical Frameworks

Contents

1 The Science of Never-Ending Learning

Open-Ended Evolution and Core Paradigms

2 Landmark Experiments

OpenAI Hide-and-Seek · DeepMind · POET · Digital Organisms

3 The Toolkit Landscape

Mesa · PettingZoo · JaxMARL · NEAT · LLM Frameworks

4 Building a Persistent Multi-Agent Ecosystem

Architecture · Rewards · Catastrophic Forgetting · Hardware

5 Scenario: Two AIs in a Never-Ending Forest

6 Scenario: AIs Sealed in a Box for 1,000 Years

7 The 2024–2026 Revolution

OMNI-EPIC · ASAL · Project Sid · Generative Agents at Scale

8 Getting Started: A Six-Week Path

9 Conclusion

01 The Science of Never-Ending Learning

Open-ended learning (OEL) refers to an AI system's capacity for autonomous, unbounded acquisition of increasingly complex knowledge and skills. The term draws from **open-ended evolution** in artificial life — the idea that a system should never settle into equilibrium, continually producing artifacts that are both **novel** and **learnable**. Google DeepMind formalized this in a landmark ICML 2024 oral paper, arguing that open-endedness is essential for artificial superhuman intelligence and that current foundation models, despite their power, are not open-ended because they rely on static training data.

The field traces to Christopher Langton, who coined "Artificial Life" in 1986 and organized the first ALife conference at Los Alamos in 1987. ALife studies "life as it could be" rather than just life as we know it, through software simulations ("soft ALife"), robotics ("hard"), and biochemistry ("wet"). The central unsolved problem remains: no artificial system has yet achieved truly unbounded open-ended evolution characteristic of biological life. Systems eventually plateau or cycle. But the behaviors that emerge before that plateau are remarkable.

Core Paradigms

Several paradigms power these simulations. **Reinforcement learning** (RL) trains agents to maximize cumulative rewards through trial-and-error interaction, using algorithms like Proximal Policy Optimization (PPO). **Multi-agent RL** (MARL) extends this to settings with multiple interacting agents — cooperative, competitive, or mixed — formalized as Stochastic Games. **Evolutionary strategies** and genetic algorithms maintain populations of candidate solutions that undergo selection, crossover, and mutation across generations. **Neuroevolution** (NEAT, HyperNEAT) evolves both the topology and weights of neural networks simultaneously. And **self-play**, as demonstrated by AlphaZero, creates an implicit curriculum where each improvement by an agent makes its opponent harder to beat.

> **Key Insight:** The most powerful mechanism discovered so far is the **autocurriculum**: when multiple agents create training challenges for each other through competition or cooperation, producing emergent complexity without any hand-designed curriculum.

02 Landmark Experiments

OpenAI's Hide-and-Seek: Six Phases of Emergent Tool Use

The 2019 paper "Emergent Tool Use From Multi-Agent Autocurricula" (Baker et al., ICLR 2020) remains the most compelling demonstration of multi-agent emergence. Teams of hiders and seekers competed in a physics-based environment with moveable boxes and ramps. Over **132.3 million episodes** (31.7 billion frames), six distinct behavioral phases emerged with no explicit programming:

Phase	Behavior	What Emerged
1	Random	Basic chasing and fleeing
2	Fort Building	Hiders construct walls from boxes
3	Ramp Exploitation	Seekers use ramps to breach forts
4	Ramp Defense	Hiders move and lock all ramps
5	Box Surfing	Seekers ride boxes over walls
6	Lock Everything	Hiders lock all objects before building

Each phase represented a qualitative strategic leap driven purely by competitive pressure.

DeepMind's Population-Based Approaches

DeepMind's Capture the Flag agents achieved human-level performance in Quake III Arena by training a population of 30 agents using population-based training (PBT). Professional game testers could only win **25% of matches** even after 12 hours of practice. AlphaStar reached Grandmaster level in StarCraft II (top 0.2% of human players) using a continually evolving "League" of agents. The XLand project trained agents across **3.4 million unique procedurally generated tasks**, producing agents that achieved zero-shot generalization to entirely novel games.

POET: Co-Evolving Environments and Agents

Uber AI Labs' Paired Open-Ended Trailblazer simultaneously evolved obstacle course environments and the agents solving them. The critical insight was **transfer**: solutions from one environment served as stepping stones for another, solving challenges that direct optimization alone could not crack. Enhanced POET (2020) produced phylogenetic trees of environments resembling natural phylogenies — a signature of genuine open-endedness.

Digital Organisms: Tierra, Avida, and Lenia

Tom Ray's **Tierra** (1991) placed self-replicating assembly-language programs in a closed computer where they competed for CPU time and memory. Without any fitness function, digital parasites evolved

that hijacked other programs' replication code, followed by host defenses, counter-parasitism, and hyper-parasitism — a cascade mirroring biological arms races.

Avida (1993–present) improved on Tierra with parallel execution and localized resources. Its landmark Nature 2003 paper showed that complex logic functions evolved by building on simpler ones through initially deleterious mutations. Avida populations of 10,000 organisms can process **20,000 generations per day**.

Lenia (2018–present) generalized Conway's Game of Life to continuous space, time, and states. Over **400 species in 18 families** have been catalogued, exhibiting self-propulsion, self-organization, self-replication, and intercommunication. Flow-Lenia (2025) added mass conservation, enabling emergent evolutionary dynamics.

03 The Toolkit Landscape

The ecosystem of tools is large but unevenly suited to persistent multi-agent worlds. Here is what matters most for a Python developer on Ubuntu Linux.

Tool	Best For	Key Advantage	GPU?
Mesa	Custom ecosystems, agent-based modeling	Pure Python, built-in visualization, active community	No
PettingZoo	Multi-agent RL environments	Standard API, integrates with RLlib/CleanRL	No
JaxMARL	High-speed MARL training	Up to 12,500x speedup via GPU	**Yes**
MAgent2	Large-scale swarms (100s–millions)	C++ core, handles massive populations	No
RLlib	Industrial-grade distributed RL	Self-play, league training, scaling	Yes
neat-python	Neuroevolution	Pure Python NEAT implementation	No
EvoJAX	GPU-accelerated evolution	Parallel ES/GA on GPU via JAX	**Yes**
Lenia	Continuous cellular automata / ALife	400+ catalogued species, GPU support	Yes
CrewAI	LLM multi-agent simulations	Local models via Ollama, YAML config	Opt.

Recommended starting stack: Mesa for ecosystem prototyping → PettingZoo for RL integration → JaxMARL when you need speed.

LLM-Based Agent Frameworks

For simulations using language models, **CrewAI** is the easiest on-ramp (supports local models via Ollama). **AutoGen** (Microsoft) provides more flexible multi-agent conversation frameworks. The Stanford **Generative Agents** code (20,600+ GitHub stars) is available as a research artifact — use its architecture (memory stream → retrieval → reflection → planning) as inspiration. For million-scale LLM agent simulations, **AgentTorch** (MIT Media Lab, AAMAS 2025) uses "LLM archetypes" to efficiently scale to 8.4 million agents.

04 Building a Persistent Multi-Agent Ecosystem

Architecture Fundamentals

A persistent multi-agent simulation requires four core components: a shared environment with state that persists across timesteps, agents with observation-action loops and memory, a learning mechanism (RL, evolution, or LLM reasoning), and checkpointing for long-running experiments.

The standard agent loop is: observe the local environment → decide on an action (via policy network, evolutionary controller, or LLM) → execute the action → receive reward/feedback → update internal model. For persistence, extend this with serializable environment state, agent memory (recurrent networks, explicit replay buffers, or LLM memory streams), and periodic checkpointing of all model weights, optimizer states, and RNG states.

Reward Shaping for Emergence

The most important design decision is reward structure. Research consistently shows that **multi-agent competition produces more complex emergent behavior than intrinsic motivation alone**. The simplest approach — survival-only reward where agents lose energy over time and must eat to live — produces the most open-ended outcomes, as demonstrated by Tierra and Polyworld.

For richer dynamics, add intrinsic curiosity bonuses using the Intrinsic Curiosity Module (Pathak et al., 2017), which rewards prediction errors about action consequences, or Random Network Distillation (Burda et al., 2018). Social influence rewards (Jaques et al., 2019) encourage coordination by rewarding agents whose actions have high mutual information with other agents' actions.

Solving Catastrophic Forgetting

> **Key Insight:** Progressive/continual learning is the central challenge for agents that must retain knowledge over long timeframes. Three main strategies exist: **Elastic Weight Consolidation** (penalizes changes to important parameters), **Progressive Neural Networks** (new columns with lateral connections — immune to forgetting), and **Generative Replay** (synthetic data from past experiences). Population-based evolution sidesteps the problem entirely.

For simulating thousands of years, the hybrid approach is most promising: use evolution for the outer loop (selecting which agent architectures survive across generations) and RL for the inner loop (within-lifetime learning). This mirrors biological evolution's relationship with individual learning.

Hardware Requirements

Setup	Capabilities
CPU only + 16GB RAM	Mesa models (thousands of agents), simple MARL with tabular methods, NEAT evolution
RTX 3080 (10GB) + 32GB RAM	PettingZoo with PPO/DQN training, JaxMARL with moderate parallelism
RTX 4080 (16GB) + 32GB RAM	JaxMARL with thousands of parallel environments, serious evolutionary experiments

RL models are typically small (hundreds to thousands of parameters), so the bottleneck is usually CPU-side environment simulation. Moving the environment to GPU via JAX eliminates this.

05 Scenario: Two AIs in a Never-Ending Forest

This scenario maps directly onto existing research. The closest implemented systems are **Neural MMO** (Suarez & Isola, 2019) — a persistent tile-based world with survival mechanics for up to 128 concurrent agents — and OpenAI's hide-and-seek environment.

Concrete Design

Build a **256×256 2D grid** using Mesa or a custom PettingZoo environment, procedurally generated with Perlin noise into forest (70%), clearing (15%), water (10%), and rocky terrain (5%). Food patches regenerate every 50 ticks, trees regrow every 200 ticks. A day-night cycle runs every 100 ticks; four seasons of 30 days each create a year. Each agent observes an 11×11 local grid, maintains energy (starting 100, losing 1/tick, gaining 20 from eating), and can move, gather, build, eat, drink, rest, and send 8-bit communication tokens. Train with PPO via self-play across 1,000 parallel environment instances.

Expected Emergent Behaviors (in approximate order)

- Independent foraging and resource discovery
- Territorial behavior and niche formation (Neural MMO showed species naturally diverge)
- Incidental cooperation during resource abundance
- Tool use and environmental modification under competitive pressure
- Communication emergence (Polyworld demonstrated evolved signaling)
- Seasonal stockpiling behavior
- Deliberate cooperation or division of labor

> **Key variable:** Whether agents cooperate or compete depends on resource availability. Abundance favors cooperation; scarcity drives competition or sophisticated division of labor.

06 Scenario: AIs Sealed in a Box for 1,000 Years

This is a digital terrarium — a closed system with fixed total energy where agents must survive through resource cycling. The design draws directly from Tierra and Avida.

Set total energy as a conserved quantity: food → agent energy → waste → decomposition → new food. Dead agents become resources for others. Set a carrying capacity of N agents maximum. Agent

lifespans are genetically determined and heritable. Reproduction requires meeting energy thresholds, with offspring inheriting parent genomes plus mutations. At 100 agent lifespans per simulated year, 1,000 years equals roughly **100,000 generations** — well within Avida's capability.

Projected Timeline

Period	Expected Dynamics
Years 1–100	Rapid adaptation to survival basics; initial die-offs followed by population stabilization
Years 100–300	Co-evolutionary arms races: parasitism, defense, hyper-parasitism (as Tierra demonstrated)
Years 300–500	Niche formation and ecological specialization; multiple coexisting strategies
Years 500–700	Complexity plateau — neural networks reach "edge of chaos" then stabilize
Years 700–1000	Punctuated equilibria: long stasis interrupted by rapid adaptation if perturbed

The critical challenge is **avoiding stagnation**. Strategies include periodic environmental perturbation, maintaining co-evolutionary pressure, dynamic task generation (as in XLand), and diversity maintenance through novelty search or quality-diversity algorithms.

07 The 2024–2026 Revolution

The most significant trend is the convergence of large language models with open-ended evolutionary algorithms. This combination produces capabilities neither approach achieves alone.

OMNI-EPIC: Foundation Models Generate Endless Curricula

OMNI-EPIC (Faldor, Zhang, Cully, Clune; ICLR 2025) has foundation models autonomously generate code specifying entirely new environments and tasks, creating an endless stream of learnable, interesting challenges for RL agents. This moves beyond curating tasks from a fixed pool to generating the tasks themselves — a major step toward true open-endedness.

ASAL: Automated Discovery of Artificial Life

Sakana AI's **ASAL** (December 2024) uses CLIP to automate ALife discovery across Boids, Particle Life, Game of Life, Lenia, and Neural Cellular Automata. It discovered Lenia lifeforms and cellular automata rules more open-ended than Conway's Game of Life. Their **Darwin Gödel Machine** (May 2025) takes this further: a self-improving coding agent that rewrites its own code using evolutionary principles, improving from 20% to 50% on SWE-bench.

Project Sid: AI Civilizations in Minecraft

Project Sid (Altera.AL, October 2024) placed 10–1,000+ autonomous LLM agents in Minecraft for multi-day runs. Thirty identical agents autonomously specialized into farmers, miners, engineers, and guards. Agents followed tax laws, voted to amend a constitution, spread religious beliefs through social influence, and formed merchant hubs with resource trading. In 4 hours, 49 agents collected roughly **320 unique Minecraft items** (one-third of all items in the game).

Scaling Generative Agents

Joon Sung Park's Stanford team progressed from 25 fictional agents in Smallville (2023) to simulations of **1,052 real people** built from 2-hour interview transcripts (November 2024). These agents replicated participants' survey responses with **85% accuracy**. The team raised $100M from Index Ventures to commercialize this as Simile. Meanwhile, **OASIS** demonstrated that emergent social phenomena like polarization only appear at scale — requiring 10,000+ agents to manifest.

08 Getting Started: A Six-Week Path

Week	Focus	Activities
1	Agent-Based Modeling	Install Mesa, complete first-model tutorial, build Wolf-Sheep predator-prey model
2	Multi-Agent RL	Install PettingZoo, run MPE environments with random then PPO agents via CleanRL
3	GPU-Accelerated Training	Clone JaxMARL, run IPPO baselines, compare speedup vs CPU-based Week 2 experiments
4	Neuroevolution	Install neat-python, evolve NN controllers, implement competitive coevolution
5	Continual Learning	Implement EWC on DQN agent, experiment with replay buffer strategies
6+	Combine Everything	Design persistent ecosystem with Mesa + RL/evolved agents, add checkpointing and W&B; logging

Essential Packages for Ubuntu

```
pip install gymnasium pettingzoo[all] supersuit stable-baselines3
pip install "mesa[rec]" neat-python
pip install jax jaxlib jaxmarl
pip install tensorboard wandb matplotlib
```

09 Conclusion

Three insights stand out from this research.

First, multi-agent competition is the single most powerful driver of emergent complexity — more effective than intrinsic motivation, curriculum design, or environmental complexity alone. If you want interesting behavior, put agents in competition.

Second, the stagnation problem remains unsolved but is being actively eroded: OMNI-EPIC's foundation-model-generated curricula, ASAL's automated ALife discovery, and the Darwin Gödel Machine's self-improving code all represent concrete progress toward systems that never stop generating novelty.

Third, the practical barriers have collapsed. Five years ago, replicating OpenAI's hide-and-seek experiment required massive compute. Today, JaxMARL can train multi-agent systems 12,500× faster on a single consumer GPU, Mesa lets you prototype ecosystem simulations in an afternoon, and

LLM-based agents can exhibit social behavior out of the box.

> The field's trajectory points toward a convergence: evolutionary algorithms selecting which foundation-model-powered agents survive, in procedurally generated environments created by other foundation models, with the entire system running on consumer GPUs at speeds that compress thousands of simulated years into hours of wall-clock time. **The tools exist. The research foundations are solid. The experiments are waiting to be run.**

Key References & Resources

Papers

Baker et al. (2020) — Emergent Tool Use From Multi-Agent Autocurricula — ICLR 2020

Jaderberg et al. (2019) — Human-level performance in Capture the Flag — Science

Vinyals et al. (2019) — Grandmaster level in StarCraft II — Nature

Wang et al. (2019) — POET: Paired Open-Ended Trailblazer — arXiv 1901.01753

Ray (1991) — Tierra: An Approach to Digital Organisms

Lenski et al. (2003) — The Evolutionary Origin of Complex Features — Nature

Chan (2018) — Lenia — Biology of Artificial Life — arXiv 1812.05433

Park et al. (2023) — Generative Agents — arXiv 2304.03442

Faldor et al. (2025) — OMNI-EPIC — ICLR 2025

MIT/Sakana (2024) — ASAL: Automated Search for Artificial Life — arXiv 2412.17799

Altera.AL (2024) — Project Sid: Many-agent simulations — arXiv 2411.00114

GitHub Repositories

Mesa — github.com/mesa/mesa

PettingZoo — github.com/Farama-Foundation/PettingZoo

JaxMARL — github.com/FLAIROx/JaxMARL

MAgent2 — github.com/Farama-Foundation/MAgent2

neat-python — github.com/CodeReclaimers/neat-python

EvoJAX — github.com/google/evojax

Lenia — github.com/Chakazul/Lenia

Avida — github.com/devosoft/avida

Generative Agents — github.com/joonspk-research/generative_agents

OMNI-EPIC — github.com/maxencefaldor/omni-epic

ASAL — github.com/SakanaAI/asal

OpenAI Multi-Agent — github.com/openai/multi-agent-emergence-environments

Voyager — github.com/MineDojo/Voyager